

Sistemas Operacionais

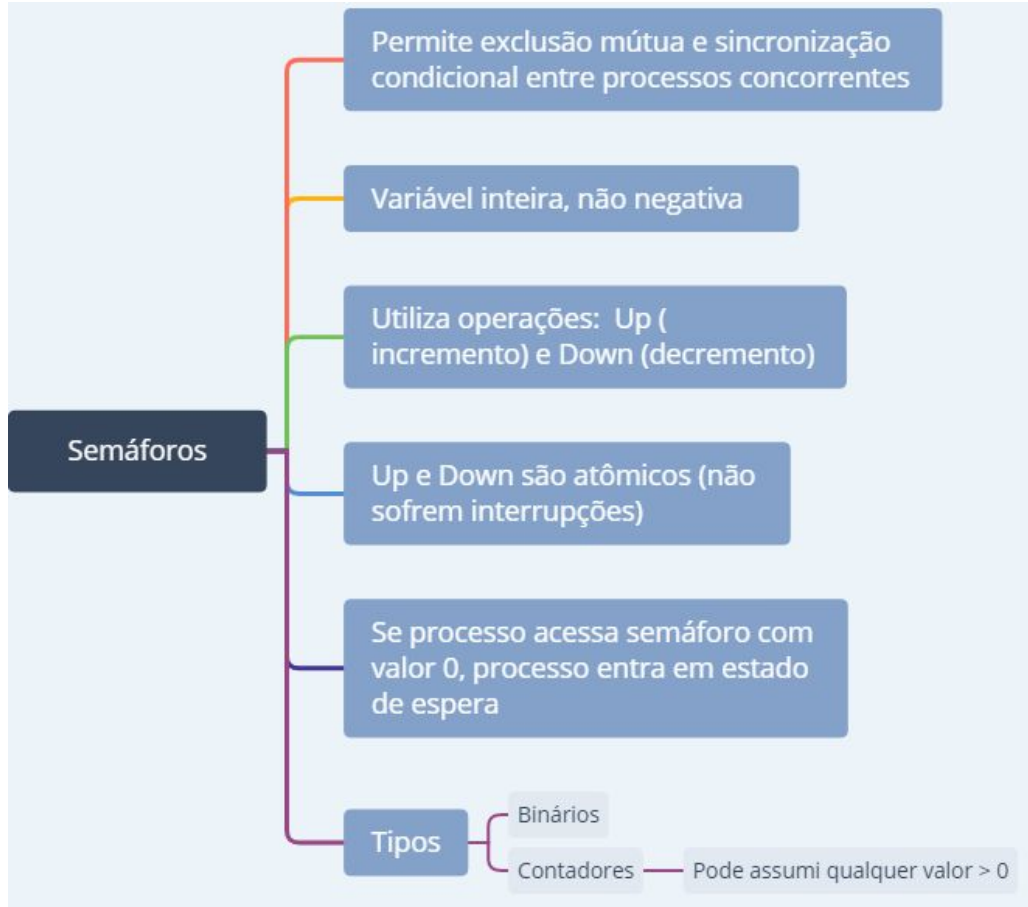
Roni Francis Shigueta

SEMÁFOROS, MONITORES, TROCA DE MENSAGENS E DEADLOCK

CONTEÚDO

- Introdução
- Semáforos
- Problemas de acesso ao recurso compartilhado
- Monitores e troca de mensagens
- Deadlock

INTRODUÇÃO



EXCLUSÃO MÚTUA UTILIZANDO SEMÁFOROS

Vantagem: não apresenta o problema da espera ocupada

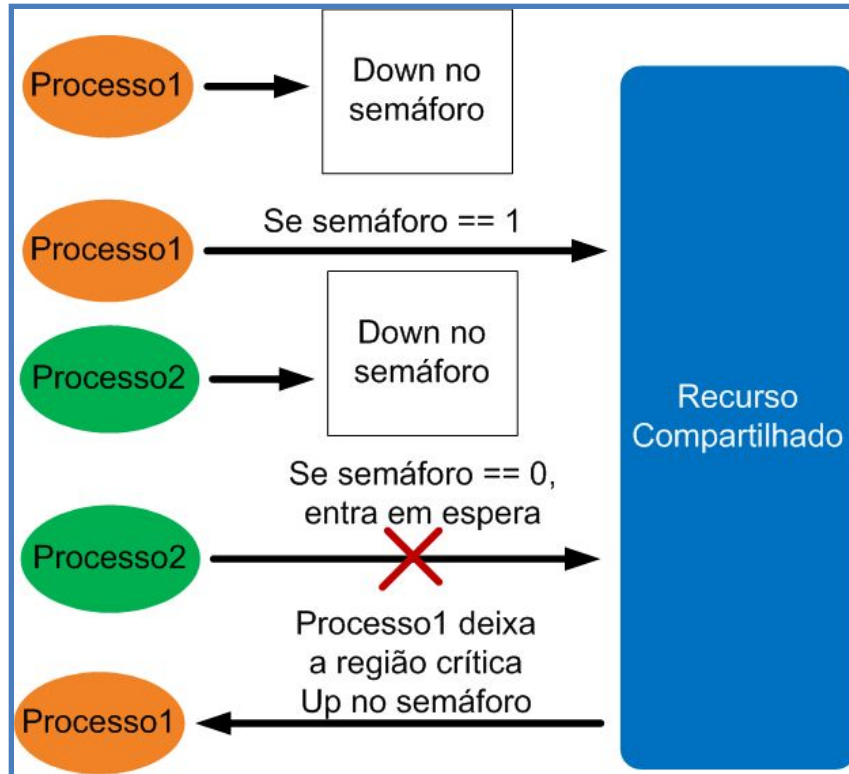


Figura 1 –Exclusão mútua com semáforos

SINCRONIZAÇÃO CONDICIONAL UTILIZANDO SEMÁFOROS

- ❑ É utilizado um semáforo contador de recursos.
- ❑ O semáforo é iniciado com o total de recursos disponíveis.
- ❑ Cada vez que um processo utiliza o recurso ele decrementa o semáforo (DOWN).
- ❑ Ao liberar o recurso, o processo incrementa o contador (UP).
- ❑ Se o valor do semáforo for zero, o processo deve entrar em estado de espera e aguarda até um novo recurso ser liberado.

PROBLEMA DO JANTAR DOS FILÓSOFOS



- Problema: hashis compartilhados
- Solução: um semáforo por filósofo. Filósofos podem ser bloqueados se não há hashis disponíveis

Figura 2 – Jantar dos filósofos

Fonte: <https://bit.ly/2XGYB8o>

PROBLEMA DO BARBEIRO DORMINHOCO



- ❑ Problema: clientes entrarem em disputa com o barbeiro.
- ❑ Solução: utilizar 3 semáforos: barbeiro, clientes, exclusão mútua.

Figura 3 – Problema do barbeiro dorminhoco

Fonte: <https://bit.ly/2XGYB8o>

MONITORES

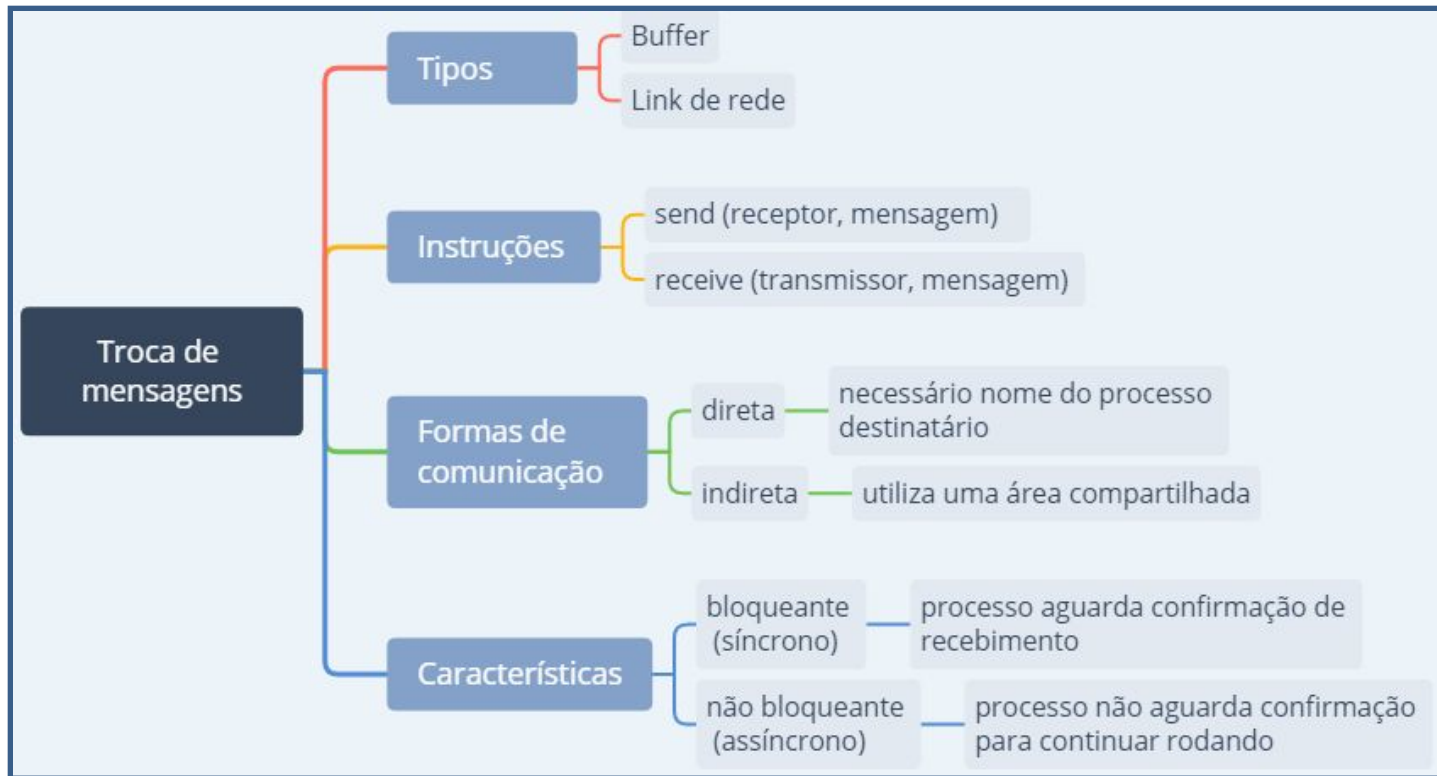
- É um mecanismo de sincronização de processos de alto nível.
- Somente um processo pode executar o monitor por vez.
- Se algum processo estiver acessando o monitor, o outro deve esperar em uma fila.
- O desenvolvedor é responsável por definir as regiões críticas como procedimentos do monitor.

Sincronização condicional utilizando monitores

- Utiliza duas instruções WAIT E SIGNAL.
- WAIT: coloca o processo em espera.
- SIGNAL: libera o processo da espera.

TROCA DE MENSAGENS

É um mecanismo de sincronização entre processos cooperativos.



DEADLOCK (IMPASSE)

De acordo com Tanenbaum (2006, p.239):

“Um conjunto de processos estará em situação de deadlock se todo processo pertencente ao conjunto estiver esperando por um evento que somente um outro processo desse mesmo conjunto poderá fazer acontecer”.

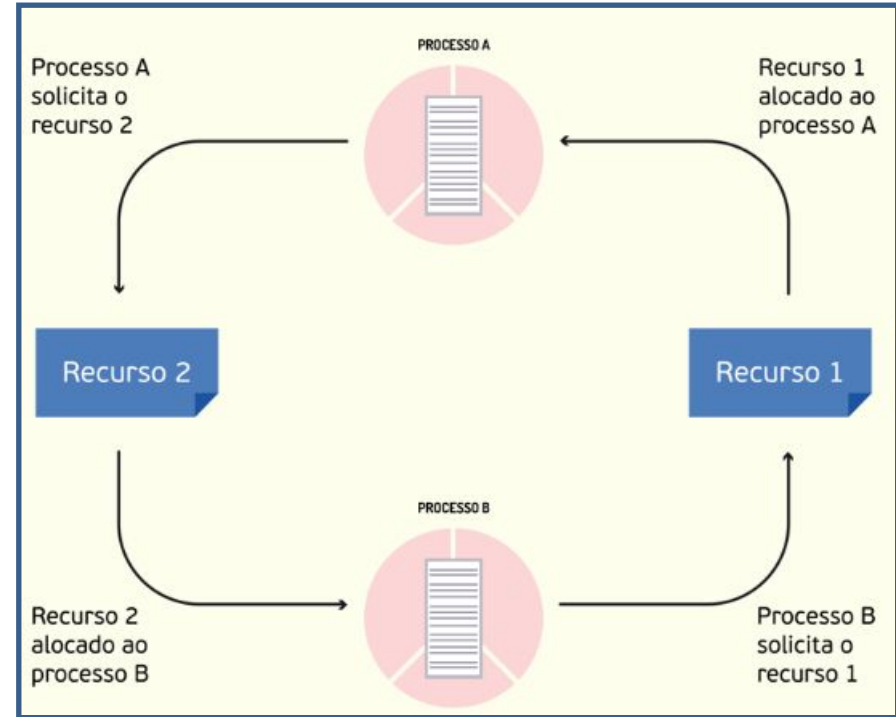


Figura 4 – Espera circular

Fonte: <https://bit.ly/2XGYB8o>

CONDIÇÕES PARA DEADLOCK



FORMAS DE TRATAR DEADLOCKS

Tratamento de deadlocks

```
graph LR; A[Tratamento de deadlocks] --> B[Detecção de deadlocks]; A --> C[Correção do deadlock]; B --> D[Necessário armazenar informações de todos os processos concorrentes e recursos disponíveis]; D --> E[recursos que cada processo aloca]; D --> F[recursos que cada processo aguarda]; C --> G[Suspender o processo e liberar os recursos alocados]; C --> H[Elimina um ou mais processos envolvidos no deadlock]; G --> I[posteriormente retorna do ponto de parada (rollback)]; H --> J[critério de eliminação: prioridade ou aleatório];
```

Detecção de deadlocks

Necessário armazenar informações de todos os processos concorrentes e recursos disponíveis

recursos que cada processo aloca

recursos que cada processo aguarda

Correção do deadlock

Suspender o processo e liberar os recursos alocados

posteriormente retorna do ponto de parada (rollback)

Elimina um ou mais processos envolvidos no deadlock

critério de eliminação: prioridade ou aleatório

REFERÊNCIAS

TANENBAUM, A. S. **Sistemas operacionais modernos.** São Paulo: Pearson Prentice Hall, 2009.

DEITEL, H. M; DEITEL, P. J.; DEITEL, D. R. C. **Sistemas Operacionais.** São Paulo: Pearson Prentice Hall, 2005.